Learning Versatile Robot Locomotion: Conditional Adversarial Imitation Learning with Disentangled Skill Conditions

Anonymous Author(s)
Affiliation
Address
email

Abstract: This work has developed a novel approach for learning diverse robot locomotion skills at scale through conditional adversarial imitation learning from unstructured motion data. Our primary contribution is the development of one single unified policy that can learn a diverse array of locomotion behaviors, including complex maneuvers such as flipping, hand-standing, galloping, and various quadruped gaits. The policy is easily controllable for both human operators and autonomous systems. We further demonstrate the policy's ability to interpolate between learned skills, thereby generating new behaviors not present in the training dataset. This capability significantly expands the behavioral repertoire of robot agents, enabling them to adapt and deal with a wider range of scenarios. By training the perceptual module as a task-level policy with visual inputs, we can extract and extend reusable skills from this unified low-level policy to complex downstream tasks, allowing the system to perform a new task such as robot parkour. Our development provides expandable hierarchical learning with a unified motor cortex encoding rich motor skills, which allows much faster training of the task-level policies that exploit the readily reusable skills for generating highly versatile capabilities of navigating and interacting in various environments.

Keywords: Imitation Learning, Quadruped Locomotion, Reinforcement Learning

1 Introduction

Legged robots offer significant advantages over wheeled robots in accessing challenging locations [1, 2]. Achieving dynamic coordination in robots to match natural agility involves complex control over underactuated mechanisms. Traditional model-based control methods enable agility, yet are limited by their reliance on detailed knowledge of the robot and environmental dynamics, and are hampered by high computational demands, sensitivity to disturbances and modeling inaccuracies [3, 4, 5, 6, 7]. Reinforcement Learning (RL) offers a promising alternative, learning optimal control policies through trial and error in varied environments [8, 9, 10]. The effectiveness of RL is challenged by the complex balance and design of reward functions, the high-dimensional nature of legged robots, which often yields narrow/unnatural skills [11, 10]. Incorporating visual observations and leveraging motion priors in RL represents significant strides toward better environmental interaction and the utilization of existing skills to develop natural and agile locomotion [12, 13, 14, 15, 16, 17, 18, 19]. Despite these efforts, previous methods have not fully captured the diversity of quadruped capabilities for learning the breadth highly agile locomotion skills. To develop general-purpose robots (humanoid or quadrupeds), they should be able to carry out a large variety of different motor skills, including locomotion and manipulation where each task involves different operational modes, each requiring a diversity of skills. Therefore, multi-skill learning becomes increasingly important towards the development of general-purpose robots.

Submitted to the 8th Conference on Robot Learning (CoRL 2024). Do not distribute.

This research focuses on multi-skill motor learning in robot locomotion. We aim to develop a generic motor cortex which encapsulates rich motor skills with all necessary fine movements that can be reused and recomposed into different locomotion skills and transitional behaviors between different skills. The research proposition is that if we are able to have a generic motor cortex that captures all the necessary fine motor skills, then reusing the pre-trained motor skills, allows solving complex downstream tasks efficiently with simple rewards, training only at the task level. The motor cortex serves as a foundational robot skill model, which can be reused by a task-level policy. This becomes an indispensable ability for scalability because it offers modularity, only training task-level policies for new and different tasks, greatly improving the data, computational efficiencies, and reward complexity.

Past works that address multi-skill learning using hierarchical structures in the form of mixture of expert (MoE) where the high-level policy switches in between discrete low-level policies [20, 21]. The convenience of training isolated policies, trades off the generic representation of rich diverse skills. The MELA architecture achieves multiple skills by dynamically synthesizing a new network through aggregated expert networks' weights and biases, preserving the richness of the motion skills[22]. However, such an architecture has no individual network that can execute independently. This naturally poses if one larger neural network can represent all the motor skills. Typically training a large neural network for diverse motor skills suffers from mode-collapse. The nature of mixed continuous dynamics and the discrete dynamics of the locomotion makes such end-to-end training even more challenging. Our research specifically addresses this issue, by proposing: (1) unsupervised pre-trained disentangled skill representations, (2) Quadruped motor policy learning with conditional adversarial rewards, (3) hierarchical discrete-continuous hybrid policy. In contrast, the typical RL framework relies heavily on reward design/regularization to induce desired behaviors, which is time-inefficient for multi-skill learning.

In particular, our unique novel contribution of pre-trained disentangled skill representations enhances the diversity of motor skills, and the contribution of discrete-continuous hybrid policy low-level motor priors eliminates the need for designing complex reward functions, streamlining the reward terms for end-goal tasks only (only four reward terms are needed in our study here).

1.1 Preliminaries and Related Works

We conceptualize the acquisition of locomotion skills through the lens of a Markov Decision Process (MDPZ [23]), represented by the tuple $M = (S, A, \gamma, p, r)$. This framework characterizes the environment by a set of possible states S and a set of possible actions A. The environment's dynamics are captured by the transition function $p(s'|s, a)$, which describes the probability of moving to a new state s' from the current state s after taking action a. With a reward function $r(s, a)$ and a discount factor γ, the goal in reinforcement learning is to devise a policy π that optimizes the agent's expected cumulative reward over time, mathematically expressed as $E\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)\right]$.

Imitation Learning (IL) teaches agents to mimic desired behaviors leveraging expert demonstration. Typically this requires complex reward functions uniquely designed for each domain. Generative Adversarial Imitation Learning (GAIL) [24] learns from expert demonstrations via adversarial rewards encouraging the agent to produce state transitions that fool a discriminator. Adversarial Movement Primitives (AMP) [17] merges goal-conditioned reinforcement learning with GAIL, controlling characters to perform tasks in a specific style given motion priors. Adversarial Skill Embedding (ASE) [25] captures a wider range of motion priors, reducing the mode-collapse issue, generating diverse skills using only local state information. Later developments like CALM [26] and C·ASE [27] improve the coverage of the motion generation by introducing skill conditioning. Alternative generative IL policies employ VAEs [28] or VQ-QAEs [29], have succeeded in similar domains, however typically require a complex tracking objective which could present issues if the demonstrations are not feasible [30, 31], or rely on a world-model [32], which introduces model inaccuracies.

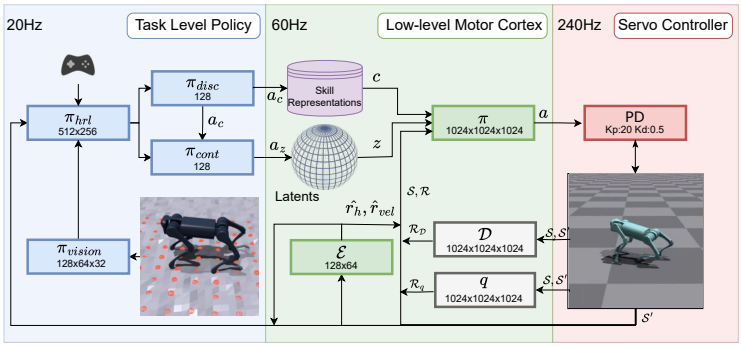


Figure 1: Hierarchical learning of perceptive motor policies. The blue section shows the high-level policy learning, where the pre-trained low-level motor policy is reused during this stage. The green section shows the reusable low-level motor policy that encapsulates rich motor skills. The red block indicates the firmware level of servo control of electric motors.

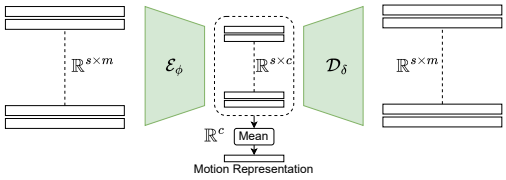


Figure 2: Training representation of diverse motions. The proposed architecture implements a β-VAE to encode diverse sequences of motion demonstrations into latent representations.

Imitation Learning for quadrupeds saw promise with [33], which achieved natural gaits through trajectory-based rewards. Yet, this lacked agility and necessitated intricate engineering of reward functions. GAIL enables the learning of distinct skills directly from demonstrations on quadrupeds, for example to perform back-flipping [34], simplifying the rewards design. Subsequent methods adopted Adversarial Movement Primitives (AMP) for learning locomotion, [35, 19, 18] learn from a variety of motion priors. These incorporate both additional regularization rewards and low-level policies are not directly transferable; learning new tasks requires retraining and new reward functions. More recent works [36] have proposed methods similar to ASE [25], but only on small motion dataset, our finding show that ASE was uncontrollable when training on more diverse dataset. Our approach diverges by learning distinct and controllable skills, while decoupling the low-level motor skills. Higher-level task policies require little training for new tasks given the base motor skills. This separation enables straightforward integration of visual inputs and task switching.

2 Methodology

This section details the techniques of the proposed method, including the low-level policy, the state estimator, skill label acquisition, data collection, the high-level policy, and the setting of simulation and training.

2.1 Low-level policy

Following C·ASE [27] the core of our low-level policy consists of the policy π, the skill encoder q and the discriminator D. The policy $\pi(a|s,z,c)$ accepts a latent variable z, skill code c and the state observation $s = \{r_h, r_{orn}, r_{vel}, r_{ang-vel}, q, \dot{q}, feet_{pos}\}$. The latents z are modeled as a 8-dimensional hyper-sphere $\mathbf{Z} = \{\mathbf{z} : \|\mathbf{z}\| = 1\}$, where samples are drawn by normalizing samples of a Normal distribution.

$$\bar{\mathbf{z}} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad \mathbf{z} = \frac{\bar{\mathbf{z}}}{\|\bar{\mathbf{z}}\|}. \quad (1)$$

We train the policy using both the imitation and diversity objectives proposed in ASE [25]. The discriminator is trained to minimise:

$$\min_{D} = -E_{d_M(s,s'|c)}\left[\log D(s,s',c)\right] - E_{d_\pi(s,s'|c)}\left[\log(1-D(s,s',c))\right] + w_{gp}E_{d_M(s,s'|c)}\left[\left\|\nabla_\phi D(\phi)\big|_{\phi\in(s,s'|c)}\right\|^2\right], \tag{1}$$

where $d_M(s,s'|c)$ and $d_\pi(s,s'|c)$ denote state transitions $(s,s'|c)$ given the class c from the dataset and the policy respectively. We pass the state transitions (s,s') for 10 consecutive steps in all cases we refer to state transitions. The last term is a gradient penalty regularization for stabilizing the training. As such we train the policy according to:

$$\arg\max_{\pi} = E_{p(Z)}E_{p(\tau|\pi,Z)}\left[\sum_{t=0}^{T-1}\gamma^t r_t\right] - \lambda E_{d^\pi(s)}E_{z_1,z_2\sim p(z)}\left[\left(\frac{D_{KL}\left(\pi(\cdot|s,z_1,c)\parallel\pi(\cdot|s,z_2,c)\right)}{0.5*(1-z_1z_2)}-1\right)^2\right] \tag{2}$$

The second term is the diversity objective, forcing action distribution to be similar where the distance between latents is small, and vice-versa. The reward r_t is comprised of two components, the discriminator reward r_D and the skill encoder reward r_q, where $r_t = r_D + r_q$.

$$r_D = -\log(1-D(s_{t-8}:s_{t+1},c)) \quad (3) \qquad r_q = \beta\log q(z_t \mid s_{t-8}:s_{t+1},c) \quad (4)$$

The skill encoder q encourages a mapping between latents z and state transitions $(s,s'|c)$, disentangling the latent space of each skill. It is modelled as a a von Mises-Fisher distribution $q(z|s,s',c) = \frac{1}{Z}\exp\left(\kappa,\mu_q(s,s',c)^T z\right)$, where μ_q is normalized, Z is a normalization constant, and k is a scaling factor. We optimize q following:

$$max_q = E_{p(z)}E_{d^\pi(s,s'|z,c)}\left[k\mu_q(s,s',c)^T z\right] \tag{5}$$

To encourage the policy to learn more challenging skills we use focal skill sampling from [27], where we sample reference skills to generate with probability $\hat{w}_c$ according to the policy ability to fool the discriminator $b_c \in (0,1)$. Where $\hat{w}_c = w_c/\sum_{c\in C} w_c$ and $w_c = (1-\alpha)w_c + \alpha\sigma(b_c)$ and $\sigma(b_c) = 1 - b_c/\sum_{c\in C} b_c$. This encourages more generation of low-performing skills enabling the policy to improve at these skills.

2.2 State Estimator

The policy state observations include un-observable features during deployments, this includes the base height and velocity. As such, we must train a state estimator $\mathcal{E}$ to predict the r_h and r_{vel} as to provide this information during deployment. The estimator receives the observable state information, the previous action a_{t-1} and the previous latents z_{t-1} e.g. $input = \{r_{orn}, r_{ang-vel}, q, \dot{q}, feet_{pos}, a_{t-1}, z_{t-1}, c_{t-1}\}$. We train this end-to-end, where predicted velocities and heights are provided as the observation during the policy roll-out and in the policy gradient computation. This causes high variance in the critic and policy gradients leading to instability, such that the policy is unable to overcome the discriminator. We resolve this using asymmetric learning, where the critic observes the true simulated values for the height and velocity terms. The state estimator is optimized with adam and a learning rate of $1e-4$.

2.3 Skill Label Acquisition

The policy receives both skill labels c and latents z, where c represents meaningful descriptions of the target skill and z represent the variations within the skill. The skill labels are learned via unsupervised learning methods. We develop, a small transformer based $\beta-$VAE[37], which we call the motion VAE. It is trained to compress motion demonstration into $\mathbb{R}^c$ space. This model is trained for 100 epochs, utilizing the Adam optimizer, with a learning rate of $4e-5$ and a weight decay of $1e-6$. Both encoder and decoder implement transformer encoders with two layers, 4 attention heads, 200 dimensional hidden size, and use 64 dimensional timestep embedding concatenated to the input and latent representations respectively.

The motion dataset is processed into the same state representation as the policy observation $\mathbb{R}^m$ and chunked into short sequences of length $\mathbb{R}^s = 200$, the length of each of our motion demonstrations. When training the motion VAE, we pass a window of 50 timesteps ($\mathbb{R}^s = 50$) over the motion sequence to the VAE, generated using sliding window with step 1. The encoder compresses motion states $\mathcal{E}_\phi : \mathbb{R}^{s\times m} \to \mathbb{R}^{s\times c}$ and decoder reconstructs the motion sequence $\mathcal{D}_\delta : \mathbb{R}^{s\times c} \to \mathbb{R}^{s\times m}$. To acquire a motion representation for the policy, the full length demonstration sequence is encoded into latent space (without sampling), we then using mean pooling over the encoded motion sequences $f : \mathbb{R}^{s\times c} \to \mathbb{R}^c$ (Fig. 2).

The above process describes extracting representations of specific motion sequences. To attain the skill representation given to the policy that describe the skill we use K-means clustering over the learned represents. The skill representation for each class of skill is the centriodal representation from the learned clusters. We run the K-means with 30 clusters.

2.4 Data collection

We enrich our dataset by integrating diverse sources, including the MANN [38] motion dataset, pre-trained policies for *parkour* [10], heading control [39], and custom developed policies for specialized movements performing bound, handstand, hind leg walking, back/front flipping. To augment the dataset and mitigate rotation biases, we invert the data across the XY plane. The MANN Dataset Contains approximately 4 minutes of unstructured motion data used, including various gaits and agile movements. We apply inverse-kinematic re-targeting [33] to adapt dog motion capture data to the A1 Unitree, this necessitates manual cleaning for anomalies like ground penetration and inverted joints. The parkour policy provides some dynamic movement demonstration such as jumping over small obstacles and varying its walking speed and turning radii.

2.5 High Level Policy

The high-level policy (Fig. 1) controls the low-level imitation policy through latent actions. The high-level policy is comprised of four components, the vision encoder π_{vision}, the backbone π_{hrl}, discrete action policy $\pi_{disc}(a_c|\pi_{hrl}(s))$, and the continuous policy $\pi_{cont}(a_z|\pi_{hrl}(s), c(\pi_{discr}(s)))$ choosing both the latents z and a discrete skill mapped to the corresponding skill representation $c(a_c)$. The continuous policy actions a_z are bounded between -1 and 1 and normalized $\hat{z} = a_z/||a_z||_2$, ensuring the latents passed to low level policy are on the surface of the unit hyper-sphere. This is a hybrid continuous-discrete policy where the continuous policy is conditioned on the output of the discrete policy where we train them jointly. As such we can formulate the action distribution as $p(a_z, a_c|s) = p(a_c|s)p(a_z|c(a_c), s)$ and use the standard PPO algorithm.

Following the terrain curriculum and state observation from [10], which includes scan-dots in the input space, providing a height-map a to our agent for visual locomotion control. The agent must learn heading and velocity control, to jump obstacles, sloped surfaces and up and down large steps. The reward for the high-level policy includes the velocity tracking, foot stumble, foot edge, and collision terms from [10], excluding all other regularization terms. Additionally, we include a reward from the low-level discriminator which helps to prevent jittery actions cause by large changes in a_z and rapid skill switching of $c(a_c)$.

2.6 Simulation

This section outlines the simulation setup and hardware specifications for our experiments. We employ the IsaacGym physics simulator to train a simulated A1 Unitree quadruped robot. For low-level policy training, the process is conducted without domain randomization on a flat surface, but we introduce noise into the state space signals to mimic physical sensor inaccuracies. We incorporate domain randomisation techniques for high-level policy training, including sensor noise, delays, external disturbances, and varied terrains with up to 8cm of height-field variation. All models are trained on a Nvidia RTX4090 with 24Gb of GPU memory, utilizing 6000 parallel environments for

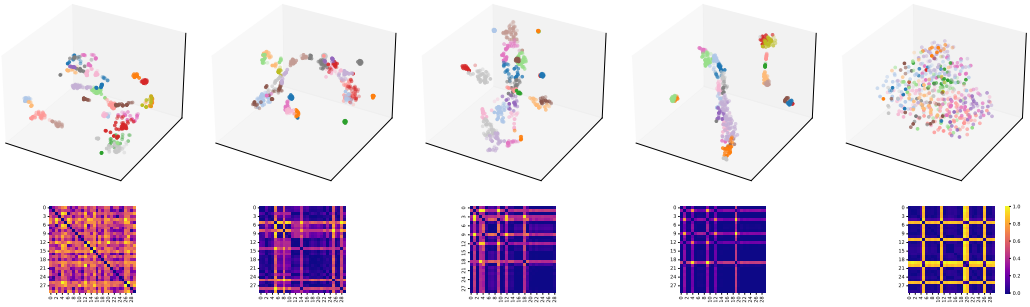


Figure 3: 3D T-SNE of motion representations, classified with k-mean, and corresponding heat-maps beneath showing the inter-cluster FID scores. More red in the heat-map indicates a greater difference in the motions between the learned clusters which is desired. Moving from the to right the columns are generated with $\beta \in \{0.1, 1.0, 2.0, 5.0, 10.0\}$.



Figure 4: 2D T-SNE of policy generated motion representations. Right assign colours by the skill condition c used at generation $\pi(\cdot|\cdot, c)$, these are the target colours for the Left plot. Left assigns colours using skill classification methods developed.

low-level and high-level policy training. The training of the low-level policy takes approximately 3 days, and the high-level policy requires around 8 hours.

3 Experimental Results

Skill Label Analysis: The motion VAE learns meaningful representations of motions as shown in the plotted 3D projected representations (Fig. 3). The dots show the motion representations while the colours show the class assigned to each motion representation using K-means clustering over 30 classes. Low β values resulted in more distinct formation of clusters which enabled improved skill classification. Higher β values resulted in more normally distributed representations, but the classification performance degraded as illustrated by the associated heat maps showing low FID scores between each cluster (Fig. 3), meaning that different skills contained similar motions. We note improved performance training the motion VAE with shorter sequences of data; training with 200 step sequences resulted in faster deterioration of the motion representation, and less distinct skills determined by the inter-cluster FID scores.

We continued further experiments, using the representations and clusters from the motion VAE trained with $\beta = 0.1$. These representation are well disentangled forming separated clusters where each class of motion lies in similar and isolated vector space. We note that observing the motion associated within each cluster show strong similarities, such as directional turning, mocap, fast vs slow, and specialized motions are clustered accordingly.

Learned Skill Analysis: The diversity of motions generated by the low-level policy exceeds the motion dataset (observed by expanding the cluster areas) and it is capable of generating a wide variety of movements, including hind-leg walking, hand-standing, galloping, rapid turning, efficient trotting (Fig. 6), and back-flipping which we display in our supplementary video.

We collect observations while running the policy, randomly sampling c and z, 5,000 times over 600 step length episodes, where each sample is executed between 150 and 200 steps, initializing in the

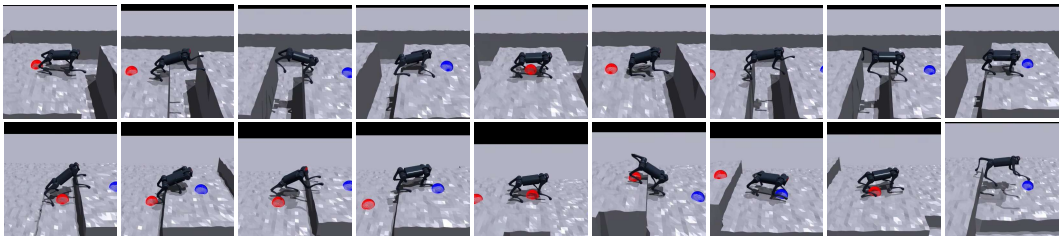


Figure 5: Down-stream deployment, this shows a time lapse the hierarchical systems navigating parkour environment over steps and gaps with 2cm heightfield perturbations.



Figure 7: Sim-to-Real: We show a time lapse of the quadruped navigating according operator generated joy-stick commands.

default pose at the start of the episode. We processes this generated data via the motion VAE and plot the 2D T-SNE motion representations (Fig. 4). It shows the motions generated (crosses) exceed the motion dataset (dots), where generated motion expand the cluster areas around the motion dataset. Furthermore, we note high similarity between both T-SNE plots (Fig. 4) visualizing the controlability derived from the skill condition, as the skill conditioning closely matches the motions generation. We can identify more mis-classifcation in areas where distinct skills are clustered more densely with others. This motivate the need for strong disentanglement for the learned cluster regions.

The accuracy of our policy is the agreement between the skill label condition policy, and the classification of the generated motion. The agreement is higher for some classes over others, with an overall classification accuracy of 58%. We note the skills with higher accuracy (12 of the 30 classes achieve over 70% accuracy) of have less similarity with other classes. This highlights the ability to have fine grained control over the executed skills. The policy additionally shows the well balanced generation capability, generating skills with similar probabilities according to the classifier, the mean of the probability of generating each skill was 3.33%, which is expected as there are 30 classes.

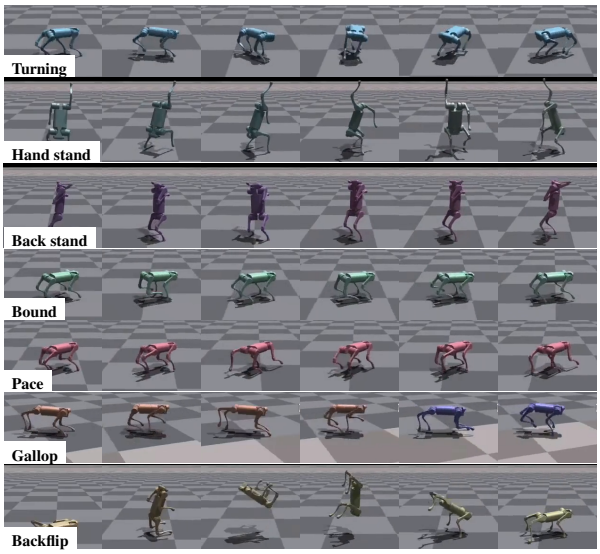


Figure 6: Time-lapse images of robot gaits.

Application for Downstream Tasks: The parkour scenario used for testing down-stream tasks demonstrates the re-usability of the low-level policy, and that we can learn complex down stream tasks which require the composition of many skills with the integration of perception. The adaptability of the low-level policy is emphasized in it successfully completing a parkour obstacle course (Fig. 7). The high-level policy adeptly handles various obstacles, including gaps, slopes, hurdles, and steps, all while adhering to specified linear and angular velocity commands to hit its target way-points, rapidly learning heading control within 100 iterations and the composition of skills is far more challenging requiring 1000 iteration to adeptly overcome small hurdle and gaps. We compare our hierarchical system against training a policy from scratch with the same rewards (excluding the discriminator rewards), our framework without the skill representations(no

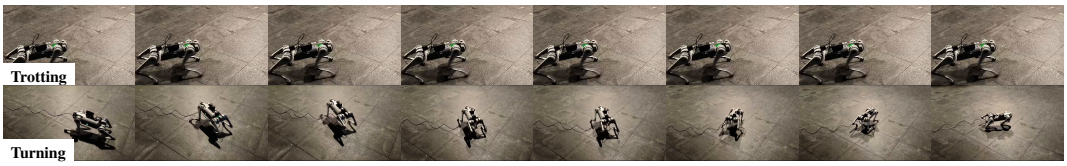


Figure 8: Sim-to-Real: We show a time lapse of the quadruped performing the trotting gait.

skill) (Table 1). We test on four of environments including gaps, hurdles, slopes and steps, with 2cm height-field perturbations, over four difficulty settings for each. The agents must follow way-points across the terrain. The Results show the mean and standard deviation of way-points reached and return acquired over ten attempts. We note our policy significantly outperforms the scratch policy on all settings and no skill policy on the majority of settings.

Table 1: Comparison of model performance with best performing polices high-lighted

Model/Task		**Gap**				**Hurdle**			
Difficulty Level		**0.2**	**0.4**	**0.6**	**0.8**	**0.2**	**0.4**	**0.6**	**0.8**
Ours	Rewards	46.5 ± 3.91	44.4 ± 3.09	42.67 ± 10.31	48.51 ± 10.25	47.6 ± 12.6	65.7 ± 9.81	65.2 ± 4.02	62.6 ± 3.21
	Goals	7.00 ± 0.00	6.72 ± 0.90	6.18 ± 1.25	6.54 ± 1.03	4.90 ± 1.64	6.72 ± 0.90	6.82 ± 0.40	7.00 ± 0.00
No Skill	Returns	42.7 ± 9.50	37.1 ± 12.0	28.4 ± 9.5	31.4 ± 8.74	30.7 ± 10.0	54.2 ± 14.0	56.5 ± 15.8	42.8 ± 21.6
	Goals	6.45 ± 1.50	5.18 ± 2.08	3.36 ± 1.68	3.81 ± 1.48	3.27 ± 1.48	5.72 ± 1.42	6.00 ± 1.48	4.72 ± 2.32
Scratch	Rewards	25.5 ± 8.9	25.0 ± 6.98	20.1 ± 1.85	20.8 ± 1.67	36.0 ± 8.53	20.5 ± 2.00	19.0 ± 1.79	21.4 ± 3.79
	Goals	2.99 ± 1.30	2.63 ± 1.03	2.00 ± 0.00	2.00 ± 0.00	2.36 ± 0.50	2.00 ± 0.00	2.00 ± 0.00	2.00 ± 0.00

Model/Task		**Slopes**				**Steps**			
Difficulty Level		**0.2**	**0.4**	**0.6**	**0.8**	**0.2**	**0.4**	**0.6**	**0.8**
Ours	Rewards	45.1 ± 8.49	33.9 ± 9.79	27.6 ± 7.61	21.31 ± 5.34	53.74 ± 6.40	54.7 ± 12.0	33.9 ± 16.0	15.68 ± 8.02
	Goals	6.63 ± 0.92	6.09 ± 1.45	3.45 ± 1.63	1.90 ± 0.53	7.00 ± 0.00	6.54 ± 1.50	3.36 ± 2.37	1.00 ± 0.00
No Skill	Rewards	50.3 ± 3.43	36.6 ± 7.0	29.24 ± 12.82	15.12 ± 3.16	47.2 ± 13.7	20.4 ± 6.32	9.62 ± 2.25	7.94 ± 3.17
	Goals	7.00 ± 0.00	6.27 ± 2.10	5.09 ± 2.21	1.90 ± 0.53	6.18 ± 1.83	2.00 ± 0.63	1.00 ± 0.00	1.00 ± 1.00
Scratch	Rewards	16.7 ± 2.58	18.2 ± 4.96	14.0 ± 3.66	18.0 ± 4.8	30.38 ± 6.61	29.1 ± 4.88	20.2 ± 1.32	12.1 ± 3.05
	Goals	2.20 ± 0.42	2.27 ± 0.90	2.09 ± 0.30	2.0 ± 0.00	6.27 ± 1.68	1.63 ± 0.50	1.00 ± 0.00	1.00 ± 0.00

Real Robot Testing: Taking the hierarchical policy further, we deploy it to the Go1 Unitree quadruped. We show it's ability to navigate the real-world according to the velocity commands, walking over complex terrains all while utilizing the low-level policy trained without any domain randomization (Fig.7), and running the low-level policy in isolation which is smoother (Fig.8). We demonstrate sim-2-sim testing examples (supplementary video) in RaiSim for more agile skills.

Low-level Limitations: During the low-level policy training, not all skills perform equally, this is mitigated by the focal skill sampling yet, we observe the certain skills remain unable to overcome the discriminator. Specifically we find the skills including front-flip demonstrations are unsuccessful, and backward walking is less elegant. Future works should seek to increase the diversity of these motion demonstrations. On real robot testing, more dynamic low-level skills resulted in highly jittery actions and failures, future works should employ domain adaption methods such as RMA [9] to address this.

4 Conclusions

In this study, we developed a versatile low-level quadruped locomotion controller that distinctly separates motor control from task-level directives, enabling agile and reliable movement. This system learns a wide array of gaits, including complex maneuvers like handstands, from a diverse unstructured collection of motion data sourced from various domains. Our findings reveal proficiency in generating skills from the dataset, and the acquisition of a disentangled latent representation. Through simulated tests, the high-level agent showcased its proficiency in traversing exceptionally challenging environments and exhibited advanced skill composition, facilitating precise velocity control and reactive navigation in response to visual stimuli of obstacles from only four reward terms. Learning to complete complex tasks with simple rewards as we demonstrate, will be crucial moving robotics forward into future modalities, where we have not discovered the required set of regularization terms for preferment locomotion control.

References

[1] J. Bhatti, A. R. Plummer, P. Iravani, and B. Ding. A survey of dynamic robot legged locomotion. In *2015 International Conference on Fluid Power and Mechatronics (FPM)*, pages 770–775, 2015. doi:10.1109/FPM.2015.7337218.

[2] M. F. Silva and J. T. Machado. A historical perspective of legged robots. *Journal of Vibration and Control*, 13(9-10):1447–1486, 2007. doi:10.1177/1077546307078276. URL https://doi.org/10.1177/1077546307078276.

[3] J. Di Carlo, P. M. Wensing, B. Katz, G. Bledt, and S. Kim. Dynamic locomotion in the mit cheetah 3 through convex model-predictive control. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1–9, 2018. doi:10.1109/IROS.2018.8594448.

[4] B. Katz, J. Di Carlo, and S. Kim. Mini cheetah: A platform for pushing the limits of dynamic quadruped control. In *2019 international conference on robotics and automation (ICRA)*, pages 6295–6301. IEEE, 2019.

[5] D. Kim, J. Di Carlo, B. Katz, G. Bledt, and S. Kim. Highly dynamic quadruped locomotion via whole-body impulse control and model predictive control. *arXiv preprint arXiv:1909.06586*, 2019.

[6] C. Nguyen, L. Bao, and Q. Nguyen. Continuous jumping for legged robots on stepping stones via trajectory optimization and model predictive control. In *2022 IEEE 61st Conference on Decision and Control (CDC)*, pages 93–99, 2022. doi:10.1109/CDC51059.2022.9993259.

[7] Q. Nguyen, M. J. Powell, B. Katz, J. D. Carlo, and S. Kim. Optimized jumping on the mit cheetah 3 robot. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 7448–7454, 2019. doi:10.1109/ICRA.2019.8794449.

[8] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter. Learning agile and dynamic motor skills for legged robots. *Science Robotics*, 4(26):eaau5872, 2019.

[9] A. Kumar, Z. Fu, D. Pathak, and J. Malik. Rma: Rapid motor adaptation for legged robots. *ArXiv*, abs/2107.04034, 2021. URL https://api.semanticscholar.org/CorpusID:235650916.

[10] X. Cheng, K. Shi, A. Agarwal, and D. Pathak. Extreme parkour with legged robots. *arXiv preprint arXiv:2309.14341*, 2023.

[11] Z. Fu, A. Kumar, J. Malik, and D. Pathak. Minimizing energy consumption leads to the emergence of gaits in legged robots. In *Conference on Robot Learning (CoRL)*, 2021.

[12] A. Agarwal, A. Kumar, J. Malik, and D. Pathak. Legged locomotion in challenging terrains using egocentric vision. In *Conference on Robot Learning*, pages 403–415. PMLR, 2023.

[13] R. Yang, G. Yang, and X. Wang. Neural volumetric memory for visual locomotion control. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1430–1440, Los Alamitos, CA, USA, jun 2023. IEEE Computer Society. doi:10.1109/CVPR52729.2023.00144. URL https://doi.ieeecomputersociety.org/10.1109/CVPR52729.2023.00144.

[14] S. Kareer, N. Yokoyama, D. Batra, S. Ha, and J. Truong. Vinl: Visual navigation and locomotion over obstacles. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 2018–2024. IEEE, 2023.

[15] L. Hasenclever, F. Pardo, R. Hadsell, N. Heess, and J. Merel. Comic: Complementary task learning & mimicry for reusable skills. In *International Conference on Machine Learning*, pages 4105–4115. PMLR, 2020.

[16] S. Bohez, S. Tunyasuvunakool, P. Brakel, F. Sadeghi, L. Hasenclever, Y. Tassa, E. Parisotto, J. Humplik, T. Haarnoja, R. Hafner, M. Wulfmeier, M. Neunert, B. Moran, N. Siegel, A. Huber, F. Romano, N. Batchelor, F. Casarini, J. Merel, R. Hadsell, and N. Heess. Imitate and repurpose: Learning reusable robot movement skills from human and animal behaviors, 2022.

[17] X. B. Peng, Z. Ma, P. Abbeel, S. Levine, and A. Kanazawa. Amp: Adversarial motion priors for stylized physics-based character control. *ACM Trans. Graph.*, 40(4), July 2021. doi:10.1145/3450626.3459670. URL http://doi.acm.org/10.1145/3450626.3459670.

[18] Y. Wang, Z. Jiang, and J. Chen. Learning robust, agile, natural legged locomotion skills in the wild, 2023.

[19] J. Wu, G. Xin, C. Qi, and Y. Xue. Learning robust and agile legged locomotion using adversarial motion priors. *IEEE Robotics and Automation Letters*, 8(8):4975–4982, 2023. doi:10.1109/LRA.2023.3290509.

[20] X. B. Peng, M. Chang, G. Zhang, P. Abbeel, and S. Levine. Mcp: Learning composable hierarchical control with multiplicative compositional policies. *Advances in Neural Information Processing Systems*, 32, 2019.

[21] E. Triantafyllidis, F. Acero, Z. Liu, and Z. Li. Hybrid hierarchical learning for solving complex sequential tasks using the robotic manipulation network roman. *Nature Machine Intelligence*, 5(9):991–1005, 2023.

[22] C. Yang, K. Yuan, Q. Zhu, W. Yu, and Z. Li. Multi-expert learning of adaptive legged locomotion. *Science Robotics*, 5(49):eabb2174, 2020.

[23] M. L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, Inc., USA, 1st edition, 1994. ISBN 0471619779.

[24] J. Ho and S. Ermon. Generative adversarial imitation learning. *Advances in neural information processing systems*, 29, 2016.

[25] X. B. Peng, Y. Guo, L. Halper, S. Levine, and S. Fidler. Ase: large-scale reusable adversarial skill embeddings for physically simulated characters. *ACM Trans. Graph.*, 41(4), jul 2022. ISSN 0730-0301. doi:10.1145/3528223.3530110. URL https://doi.org/10.1145/3528223.3530110.

[26] C. Tessler, Y. Kasten, Y. Guo, S. Mannor, G. Chechik, and X. B. Peng. Calm: Conditional adversarial latent models for directable virtual characters. In *ACM SIGGRAPH 2023 Conference Proceedings*, pages 1–9, 2023.

[27] Z. Dou, X. Chen, Q. Fan, T. Komura, and W. Wang. C·ase: Learning conditional adversarial skill embeddings for physics-based characters. In *SIGGRAPH Asia 2023 Conference Papers*, SA '23, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400703157. doi:10.1145/3610548.3618205. URL https://doi.org/10.1145/3610548.3618205.

[28] D. P. Kingma and M. Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.

[29] A. van den Oord, O. Vinyals, and K. Kavukcuoglu. Neural discrete representation learning, 2018.

[30] H. Yao, Z. Song, Y. Zhou, T. Ao, B. Chen, and L. Liu. Moconvq: Unified physics-based motion control via scalable discrete representations, 2023.

[31] Q. Zhu, H. Zhang, M. Lan, and L. Han. Neural categorical priors for physics-based character control, 2023.

[32] J. Won, D. Gopinath, and J. Hodgins. Physics-based character controllers using conditional vaes. *ACM Trans. Graph.*, 41(4), jul 2022. ISSN 0730-0301. doi:10.1145/3528223.3530067. URL https://doi.org/10.1145/3528223.3530067.

[33] X. B. Peng, E. Coumans, T. Zhang, T.-W. Lee, J. Tan, and S. Levine. Learning agile robotic locomotion skills by imitating animals. *arXiv preprint arXiv:2004.00784*, 2020.

[34] C. Li, M. Vlastelica, S. Blaes, J. Frey, F. Grimminger, and G. Martius. Learning agile skills via adversarial imitation of rough partial demonstrations, 2022.

[35] E. Vollenweider, M. Bjelonic, V. Klemm, N. Rudin, J. Lee, and M. Hutter. Advanced skills through multiple adversarial motion priors in reinforcement learning. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5120–5126, 2023. doi:10.1109/ICRA48891.2023.10160751.

[36] C. Li, S. Blaes, P. Kolev, M. Vlastelica, J. Frey, and G. Martius. Versatile skill control via self-supervised adversarial imitation of unlabeled mixed motions. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 2944–2950, 2023. doi:10.1109/ICRA48891.2023.10160421.

[37] I. Higgins, L. Matthey, A. Pal, C. Burgess, X. Glorot, M. Botvinick, S. Mohamed, and A. Lerchner. beta-VAE: Learning basic visual concepts with a constrained variational framework. In *International Conference on Learning Representations*, 2017. URL https://openreview.net/forum?id=Sy2fzU9gl.

[38] H. Zhang, S. Starke, T. Komura, and J. Saito. Mode-adaptive neural networks for quadruped motion control. *ACM Trans. Graph.*, 37(4), jul 2018. ISSN 0730-0301. doi:10.1145/3197517.3201366. URL https://doi.org/10.1145/3197517.3201366.

[39] GitHub - unitreerobotics/unitree_rl_gym — github.com. https://github.com/unitreerobotics/unitree_rl_gym. [Accessed 07-06-2024].